ECS 408/608 : Operaitng System
Operation on Process

Instructor: Dr. Sukarn Agarwal,
Assistant Professor,
EECS Department,
IISER Bhopal

January 30, 2026

Abstract

In this assignment, you have to learn a parent and child creation and termination process using `fork()` and `exec()` wrapper functions that we learned in class. All the assignment problems are implemented in C/C++ program language. You can use any OS to execute this assignment. We have seen the code snippets of Windows and Linux-based systems in the class.

Prerequisite Process creation involves the following steps:

- Parent process creates a child process, which creates another set of child processes, forming a tree of processes.
- Each child process is either a duplicate of the parent process or loads a new program.

Basic Unix/Linux uses system call functions to implement the process parent/child process.

- **fork()** system call creates new process
- **exec()** system call used after a fork() to replace the process' memory space with a new program
- Parent process calls **wait()** waiting for the child to terminate

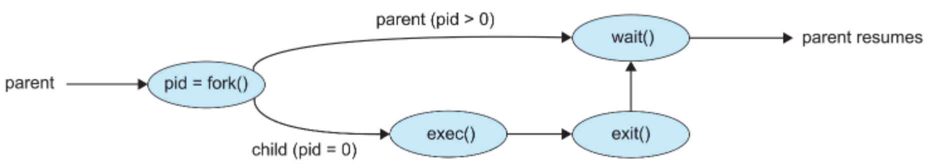

Figure 1: Parent Child Creation Process Overview

Problems

1. Implement a C/C++ program that represents a parent-child relationship between two processes. Parent and child processes do print tasks, which are as follows:
   (a) **Parent:**
      i. Hi! I am a parent process.
      ii. My PID is <**PID**>.
      iii. My Child PID is <**PID**>.
   (b) **Children:**
      i. Hi! I am a child process.
      ii. My PID is <**PID**>.
      iii. MY Parent PID is <**PID**>.

2. Implement a C/C++ program with parent-child process relations that accept n integers (Note that N can be anything based on user input) as input. The parent process sorts these integers using merge sort, whereas the child process sort these integers using quick sort and prints the respective result in the format as follows:
   (a) **Parent:**
      i. Hi! I am a parent process.
      ii. My PID is <**PID**>.
      iii. Merge Sort Result: <Sorted Result>.
   (b) **Children:**
      i. Hi! I am a child process.
      ii. My PID is <**PID**>.
      iii. MY Parent PID is <**PID**>.
      iv. Quick Sort Result: <Sorted Result>.

3. Write a C program with a parent-child relationship and make the child process orphan by terminating the parent process earlier than the child. The parent process performs the task of calculating the factorial of n numbers (User Input). Whereas the task performed by the child process is to generate the m (User-based input) Fibonacci numbers. Note that $m >> n$. The output format is as follows:
   (a) **Parent:**
      i. Hi! I am a parent process.
      ii. My PID is <**PID**>.
      iii. Factorial for number 5: 120.
      iv. I am terminated earlier than my child.
   (b) **Children:**
      i. Hi! I am a child process.
      ii. My PID is <**PID**>.
      iii. MY Parent PID is <**PID**>.

iv. Fibonacci numbers for 10 numbers: $0, 1, 1, 2, 3, 5, 8, 13, 21, 34$.

4. Write a C program showing the parent-child relationship with tree structure as shown in Figure 2. The process prints a message to verify the parent-child relationship. The sample output format is as shown below:

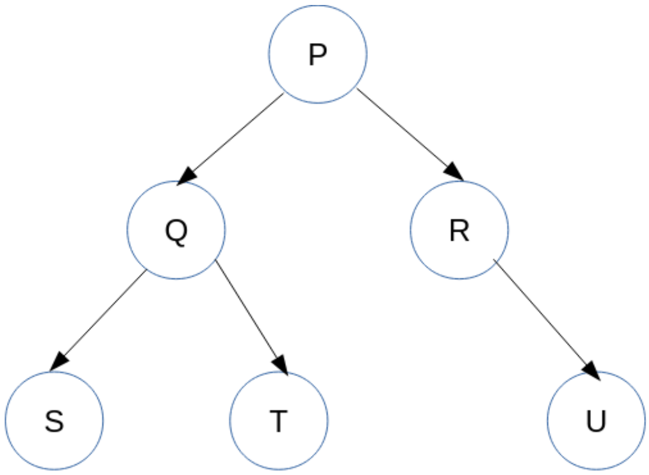

Figure 2: Parent Child Tree Relationship

(a) **P:**
   i. Hi! I am a root process.
   ii. My PID is <**PID**>.

(b) **Q:**
   i. Hi! I am a child process Q of Parent P.
   ii. My PID is <**PID**>.
   iii. MY Parent PID is <**PID**>.

(c) **R:**
   i. Hi! I am a child process R of Parent P.
   ii. My PID is <**PID**>.
   iii. MY Parent PID is <**PID**>.

(d) **S:**
   i. Hi! I am a child process S of Parent Q.
   ii. My PID is <**PID**>.
   iii. MY Parent PID is <**PID**>.
   iv. My Grand Parent PID is <**PID**>. (Optional)

(e) **T:**
   i. Hi! I am a child process T of Parent Q.
   ii. My PID is <**PID**>.
   iii. MY Parent PID is <**PID**>.
   iv. My Grand Parent PID is <**PID**>. (Optional)

(f) **U:**

   i. Hi! I am a child process U of Parent R.
   ii. My PID is <**PID**>.
   iii. MY Parent PID is <**PID**>.
   iv. My Grand Parent PID is <**PID**>. (Optional)

Assignment Submission Instruction:

- Based on your observed result, create a report with your name and roll number with assignment ID (For example, Sukarn_Agarwal_21056_Assign2.pdf).
- Submitted pdf file contains answers to all the questions above in order. **Any out-of-order answer results in a zero mark.**
- With each question, attach all the screenshots you observed on your screen. Make sure that your name should appear there. **If TA finds any discrepancy in this, zero marks will be awarded.**
- **The last date to submit the assignment is by the end of day of 13 Feb 2026 (IST). Any late submission results in the 0 Marks. There are no requests to be entertained this time.**